

Projet 2 Machine Learning

Toxic Comment Classification

Orlane Bochart, Pauline Dumas, Simon Labracherie, Emeline Lavaux

3 Octobre 2025

Part 1

From Project 1 to Project 2

Part 1 — Best ML classical model from Project 1

- ▶ **BoW** : uni+bigram (1,2) combined with a **Linear SVM** classifier
- ▶ **Strengths** :
 - ▶ Simple and transparent → easy to explain why a comment is flagged (specific n-grams)
 - ▶ Fast to train (in seconds)
- ▶ **Weaknesses** :
 - ▶ Limited context understanding → relies on word frequency so struggles with nuances

Part 1 — Baseline

SVM Classifier

Class	Precision	Recall	F1-score	Support
non-toxic	0.979	0.979	0.979	28 856
toxic	0.8	0.799	0.799	3 059
<i>accuracy</i>			0.962	31 915
<i>macro avg</i>	0.889	0.889	0.889	31 915
<i>weighted avg</i>	0.962	0.962	0.962	31 915

ROC-AUC : 0.9776 PR-AUC : 0.8841

Part 2

Word embeddings

Part 2 — Method

- ▶ **Pretrained MiniLM sentence embeddings** (384 dimensions) to represent comments instead of Bag-of-Words
- ▶ **Light cleaning only** (MiniLM trained on raw, noisy text)
- ▶ Provides **semantic robustness**
- ▶ Train two classifiers on embeddings :
 - ▶ Logistic Regression (probabilities, threshold tuning)
 - ▶ Linear SVM (strong baseline, margin-based)
- ▶ **Goal** : Test whether richer embeddings **improve** performance over simpler text features

Part 2 — Light Cleaning

```
EMAIL_RE = re.compile(r'(?i)\b[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}\b')
MENTION_RE = re.compile(r'(?<!\S)@\w+')
DEOB = str.maketrans({'@': 'a', '4': 'a', '1': 'i', '|': 'i', '3': 'e', '0': 'o', '$': 's', '5': 's', '7': 't', '€': 'e', '£': 'l'})

def light_clean(s: str) -> str:
    s = unicodedata.normalize('NFKC', str(s))
    s = re.sub(r'https?:\/\/\S+|www.\S+', ' ', s)
    s = EMAIL_RE.sub(' ', s)
    s = MENTION_RE.sub(' ', s)
    s = s.translate(DEOB) # mild deobfuscation only
    return s.strip()
```

Figure – Light Cleaning

MiniLM is trained on noisy text → robust semantics with minimal cleaning

Part 2 — Results and Cost

- ▶ **Performance** : MiniLM achieves **ROC-AUC ≈ 0.96**
- ▶ **Encoding cost** : ~ 200 seconds for sentence embeddings
- ▶ **Training cost** :
 - ▶ Logistic Regression : ~ 5 seconds
 - ▶ Linear SVM : ~ 17 seconds
- ▶ **Deployment implications** :
 - ▶ Feasible if embeddings are pre-computed
 - ▶ Slower for real-time encoding
 - ▶ Bag-of-Words remains much lighter but less semantic

Part 2 — Comparison : BoW vs MiniLM (Linear Models)

Model	Toxic F1	Macro F1	ROC-AUC	PR-AUC (toxic)
BoW + Logistic Regression	0.74	0.86	0.972	0.856
BoW + Linear SVM	0.80	0.89	0.976	0.880
MiniLM + Logistic Regression	0.64	0.79	0.9625	0.8096
MiniLM + Linear SVM	0.65	0.80	0.9633	0.8139

Figure – BoW vs MiniLM

Takeaway : **BoW** performs strongly with linear models (higher F1), while **MiniLM embeddings** cost more to encode but offer **better semantic robustness** for advanced architectures.

Part 3

Transformers

Part 3 — Why Transformers

- ▶ **Capture global context** using self-attention
- ▶ Better than static embeddings or n-grams

Part 3 — Model Used

- ▶ In project 1 : **DistilBERT**
- ▶ In project 2 : **RoBERTa-base**
 - ▶ Optimized version of BERT
 - ▶ Pretrained on more data, longer sequences, larger batches
 - ▶ Uses **dynamic masking** → more robust representations

Part 3 — Results

DistilBERT

Class	Precision	Recall	F1-score	Support
non-toxic	0.984	0.980	0.982	28 856
toxic	0.821	0.852	0.836	3 059
<i>accuracy</i>			0.968	31 915
<i>macro avg</i>	0.903	0.916	0.909	31 915
<i>weighted avg</i>	0.969	0.968	0.968	31 915

ROC-AUC : 0.9864

RoBERTa-base

Class	Precision	Recall	F1-score	Support
non-toxic	0.985	0.981	0.983	28 856
toxic	0.826	0.858	0.841	3 059
<i>accuracy</i>			0.969	31 915
<i>macro avg</i>	0.905	0.919	0.912	31 915
<i>weighted avg</i>	0.970	0.969	0.969	31 915

ROC-AUC : 0.9882

- ▶ F1-score (toxic class) : 0.83 $\rightarrow$ 0.84
- ▶ ROC-AUC : 0.986 $\rightarrow$ 0.988

Part 3 — Comparison with Classical ML

► **Advantages :**

- High performance
- Ability to capture context

► **Drawbacks :**

- Computationally expensive

Model	Training time
Classical ML	Few seconds
DistilBERT	~ 40 min
RoBERTa-base	~ 2 h

- Black-box nature → less transparency than classical ML

Part 4

Prompt engineering

Part 4 — Model Used

- ▶ In project 2 : **FLAN-T5**
 - ▶ Sequence-to-sequence (encoder–decoder) model
 - ▶ Instruction-tuned version of T5 → better at following prompts
 - ▶ Pretrained on large and diverse datasets
 - ▶ Generates text token by token → can solve many tasks (classification, translation, summarization, etc.)

Part 4 — Results

FLAN-T5

Class/Model	Precision	Recall	F1-score
Zero-shot			
non-toxic	0.9838	0.9897	0.9868
toxic	0.8975	0.8466	0.8713
macro avg	0.9407	0.9181	0.9290
Few-shot			
non-toxic	0.0000	0.0000	0.0000
toxic	0.0958	1.0000	0.1749
macro avg	0.0479	0.5000	0.0875
Role prompting			
non-toxic	0.9842	0.9886	0.9864
toxic	0.8881	0.8499	0.8686
macro avg	0.9361	0.9193	0.9275

- ▶ Best config : **Zero-shot** (Macro-F1 = 0.93)
- ▶ Few-shot unstable → bad results

Part 4 — Comparison with RoBERTa

Model used : FLAN-T5 (seq2seq, T5 type)

▶ **Advantages :**

- ▶ Flexible : can solve many NLP tasks (classification, summarization, translation, etc.)
- ▶ Instruction-tuned → adapts well to prompts

▶ **Drawbacks :**

- ▶ More computationally expensive than RoBERTa (encoder + decoder vs encoder only)
- ▶ Indirect for classification (requires generation of “toxic” / “non-toxic” instead of direct probability)

Part 5

Evaluation

Part 5 — Model Comparison

Classification report (Test)

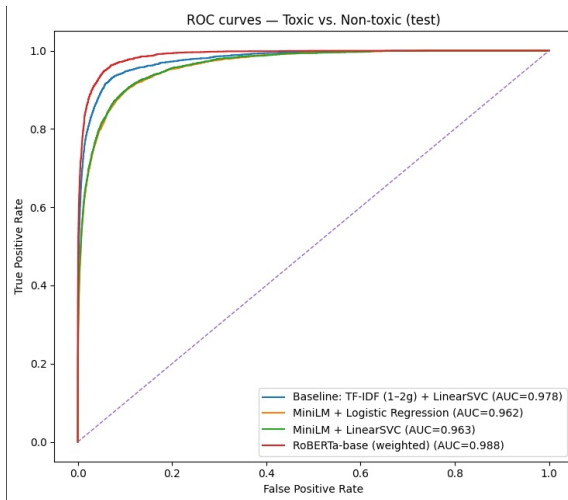
Model	Non-toxic Precision	Non-toxic Recall	Non-toxic F1-score	Toxic Precision	Toxic Recall	Toxic F1-score	Macro-F1	ROC-AUC	PR-AUC(toxic)
RoBERTa-base(weighted)	0.984	0.983	0.983	0.839	0.848	0.843	0.913	0.9881	0.9243
Baseline TF-IDF(1-2g)+ LinearSVC	0.979	0.979	0.979	0.800	0.799	0.800	0.889	0.9776	0.8841
MiniLM + LinearSVC	0.987	0.909	0.946	0.507	0.888	0.646	0.796	0.9633	0.8139
MiniLM + Logistic Regression	0.987	0.908	0.946	0.504	0.887	0.643	0.794	0.9625	0.8096
Zero-shot	0.9838	0.9897	0.9868	0.8975	0.8466	0.8713	0.9290		
Few-shot	0.0000	0.0000	0.0000	0.0958	1.0000	0.1749	0.0875		
Role prompting	0.9842	0.9886	0.9864	0.8881	0.8499	0.8686	0.9275		

Top 3 models :

1. Zero-shot
2. Role prompting
3. RoBERTa-base (weighted)

- Prompting works well thanks to huge LLMs, while RoBERTa is smaller but stable and deployable.

Part 5 — ROC Curves



RoBERTa-base reaches **ROC-AUC $\approx$ 0.988**

Part 6

Reflection

Part - 6 : Cost

- ▶ **Embeddings** : 3-4min, scalable, good trade-off time/quality.
- ▶ **Transformers** : 2h training (using GPU), inference slower : better for batch processing.
- ▶ **LLMs** : 6–7h training, 1h30 classification per prompt.
- ▶ **Deployment** : choice depends on balance between computational cost and performance.

Part - 6 : Bias

- ▶ Come from the **training data**
- ▶ E.g. : a neutral sentence like "*She is a woman*" may be flagged as toxic
- ▶ Risks for Wikipedia : moderation tool less reliable and unjustified censorship
- ▶ Ethical risks of using embeddings / transformers for moderation

Part - 6 : Transparency

- ▶ **Classical models (BoW, LR)** : easiest to explain : each word has a weight, easy to show influence, good interpretability.
- ▶ **Transformers (RoBERTa)** : harder to explain (like a black-box) : rely on contextual embeddings and multi-layer attention.
- ▶ **Prompting with LLMs** : hardest : results depend not only on internal representations but also on prompt wording, less reproducible.

Part - 6 : Privacy

- ▶ User comments are sent to an external platform for storage and analysis.
- ▶ **Risks for Wikipedia** : exposure to legal constraints (e.g., GDPR), loss of confidentiality, reduced control/sovereignty, and potential decline in user trust and platform usage.

Conclusion

- ▶ Classical models remain simple and fast, but recent approaches (Transformers, LLMs) provide contextual understanding of language and significantly higher performance. The choice of model therefore depends on the balance sought between accuracy, computational cost, explainability, and deployment constraints.